

Face Off

[MASTER BUILD PROMPT — Face Off \(paste into Claude Code\)](#)

- [1. Non-negotiable engineering constraints](#)
- [2. Project scaffold & dependencies](#)
- [3. Folder structure](#)
- [4. Design system implementation](#)
- [5. App shell — 3-tab floating nav](#)
- [6. Onboarding & Auth](#)
- [7. Play tab — matchmaking & lobby](#)
- [8. Duel feature — the game engine \(core of the app, build this carefully\)](#)
- [9. Friends tab](#)
- [10. Profile tab](#)
- [11. Monetization — RevenueCat integration](#)
- [12. Offline & connectivity handling](#)
- [13. Performance pass \(do this after everything above works functionally\)](#)
- [Definition of done](#)

MASTER BUILD PROMPT — Face Off (paste into Claude Code)

Repo: https://github.com/TheCodeDaniel/project_face_off

You are building **Face Off**, a Flutter mobile party game where two players duel using real-time face-gesture tracking. Read this entire prompt before writing any code. Where a decision isn't specified, choose the option most consistent with the architecture rules in Section 1 and state your assumption in a code comment — do not silently guess on anything touching game rules, money, or auth.

Work in this order: Section 2 (scaffold) → Section 4 (design system) → Section 5 (app shell) → Section 6 (auth/onboarding) → Section 7 (Play/matchmaking) → Section 8 (game engine — this is the core, take your time) → Section 9 (Friends) → Section 10 (Profile) → Section 11 (monetization/ads) → Section 12 (offline handling) → Section 13 (performance pass). Commit after each section with a clear message. Do not move to the next section until the current one compiles, runs, and (where applicable) its unit tests pass.

1. Non-negotiable engineering constraints

These apply to every single file you write, no exceptions:

- Feature-First Clean Architecture**, per <https://medium.com/@remy.baudet/feature-first-clean-architecture-for-flutter-246366e71c18> — each feature is self-contained under `lib/features/<feature_name>/` with its own `data/`, `domain/`, and `presentation/` subfolders. Shared code only lives in `lib/core/`. A feature must never directly import another feature's internals — cross-feature communication goes through `lib/core/` contracts or providers.
- No class or file exceeds 300 lines**. If a widget build method or a class is trending past that, extract sub-widgets into their own files under that feature's `presentation/widgets/` folder, named descriptively (not `part1`, `part2`). This is a hard rule, check it before finishing every file.
- dart format line length is forced to 120** across the whole project. Add this to `analysis_options.yaml` and to a `dart_format` config so CI and local formatting agree. Run `dart format --line-length=120` before every commit.
- State management: Riverpod**. Prefer `AsyncNotifier` / `Notifier` over legacy `StateNotifier`. No `setState` outside of purely local, ephemeral widget-only state (e.g., a text field's focus glow) that never needs to be read elsewhere.
- const everywhere it's legal**. Run `dart fix --apply` for `prefer_const_constructors` and related lints as part of your workflow, not as an afterthought.
- Avoid unnecessary rebuilds**. Use `Consumer` / `ref.watch` scoped as tightly as possible — never wrap an entire screen in one `Consumer` if only a small subtree depends on the watched value. Wrap independently-animating subtrees (score counters, avatar reactions, countdown timers) in `RepaintBoundary`.
- Isolates for heavy computation**. Any per-frame gesture math (blendshape thresholding, motion-delta calculation across pose/face landmark frames) must run via `compute()` or a long-lived `Isolate` — never block the UI isolate with this math.
- Every feature ships with tests**. Domain logic (especially the game state machine in Section 8) needs unit tests. Widgets get at minimum a smoke test (renders without throwing). This is not optional polish, it is part of "done" for every section.

2. Project scaffold & dependencies

Set up (or verify, since the repo already exists) the following:

- Flutter stable channel, null-safety, latest stable SDK constraint.
- State/DI: `flutter_riverpod`, `riverpod_annotation`, `riverpod_generator`, `build_runner`.
- Auth: `firebase_auth`, `google_sign_in`, `sign_in_with_apple`, `firebase_core`.
- Realtime/backend: `cloud_firestore` (or `firebase_database` for the low-latency signaling channel — pick `firebase_database` for the actual game-state signaling path since it's lower overhead for high-frequency small writes, and `cloud_firestore` for everything else: profiles, friends, match history, cosmetics ownership).
- Gesture tracking: integrate MediaPipe Face Landmarker via platform channel (native Android/iOS implementation) or an existing well-maintained Flutter wrapper if one meets the blendshape-output requirement — verify blendshape output (`browInnerUp`, `jawOpen`, `mouth-curvature`) is actually exposed before committing to a package; if not, write the platform channel yourself under `lib/core/gesture_engine/`.
- Onboarding: `showcaseview` (<https://pub.dev/packages/showcaseview>).
- Animation: `lottie`.
- Monetization: `purchases_flutter` (RevenueCat SDK), plus RevenueCat Ads integration per their current Flutter documentation — verify the exact package/API name against current RevenueCat docs, do not assume a name.
- Connectivity: `connectivity_plus`.
- Codegen/OTA: Shorebird is a separate CLI/tooling layer, not a pub dependency — set up `shorebird.yaml` per Shorebird's current Flutter guide.
- Testing: `flutter_test`, `mocktail`, `riverpod_test` tooling as needed.

Add `analysis_options.yaml` with strict lints (`flutter_lints` extended) plus the 300-line and 120-column rules noted above (line length via `dart_style` /formatter config, file length via a custom lint or a documented manual-check habit if no lint enforces it directly — note this limitation in a code comment if tooling can't enforce it automatically).

3. Folder structure

```
lib/
  core/
    theme/          -- design tokens, ThemeData, gradients
    widgets/        -- GradientScaffold, CoinBadge, PodiumLeaderboard,
                      RoomCard, PinCodeEntry, ActivityToast,
                      CollapsiblePanel, StatTile, DuelVsTransition,
                      PrimaryPillButton, SecondaryPillButton
    gesture_engine/  -- isolate-based gesture math, platform channel
    connectivity/    -- offline detection provider + bottom sheet
    router/          -- go_router or equivalent, route guards
    utils/
  features/
    onboarding/
      data/ domain/ presentation/
    auth/
      data/ domain/ presentation/
    app_shell/       -- the 3-tab nav shell itself
      presentation/
    play/            -- random matchmaking entry + lobby
      data/ domain/ presentation/
    duel/            -- the actual live match/game engine feature
      data/ domain/ presentation/
    friends/
      data/ domain/ presentation/
    profile/
      data/ domain/ presentation/
    monetization/    -- RevenueCat wiring, paypal, cosmetics store
      data/ domain/ presentation/
  main.dart
```

4. Design system implementation

Build every component listed in Blueprint Section 3 as its own file under `core/widgets/`, each under 300 lines, each with a preview/demo usage comment at the top. Implement both palettes (lobby + match) as named `ThemeExtensions` so screens can pull `Theme.of(context).extension<LobbyPalette>()` / `<MatchPalette>()` rather than hardcoding colors anywhere. No screen-level file should contain a raw hex color — everything routes through the theme.

`DuelVsTransition` deserves extra care: it's the dramatic pre-match beat (Blueprint Section 1, Image 6 reference) — two avatars/hands sliding in from opposite edges, colliding at a "VS" mark, camera-shake-style micro animation, countdown starting immediately after. Build this as its own isolated widget with a controllable `AnimationController` so the duel feature can trigger it precisely when both players are confirmed connected.

5. App shell — 3-tab floating nav

Build the persistent shell: `Play`, `Friends`, `Profile`, each a full nested navigator (so back-stack per tab is preserved, matching standard mobile app behavior). The bottom nav itself:

- Floating pill shape, not edge-to-edge, sits above `SafeArea` bottom inset.
- Listens to a scroll-notification stream from the active tab's scrollable.
- On scroll-down delta beyond a small threshold: animate opacity + vertical offset to hide (slide down + fade over ~200ms).
- On scroll-up, or on scroll idle for 600ms with no further scroll events: animate back to visible.
- Never hide while a modal/bottom-sheet/dialog is open on top of it.
- Selected tab indicator uses the lobby palette accent; icons + labels, not icon-only (accessibility + clarity).

6. Onboarding & Auth

First-launch experience: a short, high-polish Lottie-driven welcome sequence (2-4 screens max, swipeable or auto-advancing) introducing the core promise ("your face is the controller") without technical detail. End on a single screen with exactly two buttons: "Continue with Google" and "Continue with Apple" (iOS shows Apple first per platform convention; Android shows Google first). No email/password path, no guest mode for v1.

Auth implementation: Firebase Auth backing both providers. On first successful sign-in, create the user's Firestore profile document (display name from provider, default avatar, zero stats, free-tier cosmetics only). On subsequent sign-ins, route straight to the app shell.

In-app onboarding (showcaseview): After first sign-in, run a `Showcase` sequence highlighting: the Play tab's quick-match button, the Friends tab's add-friend action, and the Profile tab's subscription entry point. Use `ShowCaseWidget` at the shell level so it can target widgets across the initial tab. Keep it to 3-4 spotlighted steps maximum — do not showcase every button on screen, that defeats the purpose. Persist "onboarding seen" in local storage (not just Firestore) so it never replays after the first session even offline.

7. Play tab — matchmaking & lobby

Home state (Play tab default view): - Live "players online now" indicator (subtle, non-alarming — this is a trust signal, not a pressure tactic). - Primary CTA: "Quick Match" (random opponent). - Secondary: "How to Play" (re-opens a condensed rules explainer, not the full onboarding showcase). - Recent match history teaser (last 3 results, win/loss, tap to see more — full history can live in Profile).

Matchmaking flow: 1. Tap Quick Match → enter a matchmaking queue state (animated waiting screen, cancel button always visible). 2. Backend pairs two waiting users (Firestore/Realtime DB queue document, first-available match — do not over-engineer skill-based matchmaking for v1, note this as a documented v2 improvement). 3. On pairing, both clients receive the match ID and transition into the `duel` feature, which owns everything from here (Section 8). 4. If no opponent found within a reasonable timeout (e.g., 20s), show a friendly retry/cancel prompt — never leave the user in an indefinite spinner.

8. Duel feature — the game engine (core of the app, build this carefully)

This section is intentionally the most detailed. Read it fully before coding.

8.1 High-level shape

No video is ever transmitted between players. Each device runs face-gesture detection **locally**. Only lightweight game-state *events* are exchanged over the signaling channel (Firebase Realtime Database). The opponent's tracked expression values drive a locally-rendered stylized avatar/mask — you see your opponent's real-time expressions rendered as an avatar, never their raw camera feed.

8.2 Match structure

- A match is **best of 5 rounds**. First to 3 round wins takes the match.
- Each round is independent; round state resets fully between rounds except the running score.
- After both players are connected and ready, play the `DuelVsTransition` (Section 4), then begin Round 1.

8.3 Round state machine

Model this explicitly as a sealed/enum-based state machine in `duel/domain/`, not as ad-hoc booleans scattered through a widget. States:

```
RoundState =  
  Neutral          -- both players calm, waiting for the cue window  
  | CueArmed       -- countdown to cue has started (hidden from player)
```

```
| CueFired          -- the fire/dodge window is open
| ResolvingRound    -- inputs received, computing outcome
| RoundResult       -- outcome displayed, brief pause before next round
| MatchResult       -- best-of-5 concluded
```

8.4 Detailed rule logic

Neutral phase: - Both clients confirm neutral face (mouth closed, brows relaxed, no crack signal) before the round can proceed. If a player is not neutral within a grace period (e.g., 3s), show a gentle on-screen nudge (“relax your face to begin”) — do not fail the match over this, just wait.

Crack detection (runs continuously, across ALL phases below, and overrides everything): - Continuously evaluate a smile-curvature/laugh signal on each local device from the moment the round enters `Neutral` through `CueFired`. - The threshold for “cracked” must be meaningfully higher than normal resting facial variation — this needs real on-device tuning during Day 1-2 validation (see Blueprint/PRD), so implement the threshold as a configurable constant, not a magic number buried in logic, so it can be tuned without a code review of the whole state machine. - The instant a player’s local crack signal fires, that client sends a `crack_detected` event with a timestamp immediately (don’t batch/debounce this one — it needs to feel instant). The receiving side ends the round immediately: the cracking player loses the round, regardless of what phase the round was in, even during `CueArmed` before the cue has fired. - If both players crack within the same small window (e.g., within 150ms of each other, accounting for network jitter), rule it a **round draw** — no one scores, replay the round. Do not let simultaneous cracks produce an arbitrary/unfair single winner.

CueArmed phase: - A random delay between 1000ms and 4000ms (inclusive), generated **server-side** (or by a designated “host” client using a seeded random agreed by both — server-side via a Cloud Function or Realtime DB server-timestamp-seeded value is cleaner and prevents either client from gaining a timing advantage by manipulating its own local random). - The cue fire event is broadcast to both clients as close to simultaneously as the signaling channel allows. Record the server-authoritative fire timestamp; all reaction-time calculations below are measured against that authoritative timestamp, not each client’s local clock, to avoid clock-skew unfairness. - **False start rule:** if either player’s “fire” gesture (mouth-open signal) is detected with a timestamp *before* the authoritative cue-fire timestamp, that player instantly and automatically loses the round the moment their early input is received — do not wait for the cue to fire before evaluating this.

CueFired phase: - The window is open. Whichever player’s “fire” gesture is detected **first** (by authoritative server-received timestamp, not client-local timestamp) is the attacker for this exchange. - The **other** player (the target) has a reaction window of approximately 400ms from the moment the attacker’s fire event is server-timestamped to register a “dodge” gesture (eyebrow raise). If their dodge event’s timestamp falls within that window, the hit is negated and the round continues in `CueFired` (both players can still act — this is not automatically a round win for the target; if the dodge cleanly negates the attack, treat it as a reset back to an active exchange rather than an instant round win for the dodger, unless design testing during build shows a clean “successful dodge = round win for the dodger” is more fun — flag this as a tunable rule, implement the version specified here first, leave a clearly marked TODO/config flag for the alternate rule so it’s a one-line change to try both during playtesting). - If no dodge event arrives within the window, the attacker wins the round outright. - If neither player fires within a reasonable overall round timeout (e.g., 8s after cue fire, to guard against a stuck/disconnected client), resolve the round as a draw and log a diagnostic event — never let a round hang indefinitely.

ResolvingRound phase: - Pure computation, no user input accepted. Determine: round winner (or draw), reason code (`fired_first`, `false_start`, `cracked`, `simultaneous_crack`, `timeout`), and the data needed to drive the recap replay.

RoundResult phase: - Display the recap: a short (2-3s) replay driven by the recorded avatar animation data (not raw video) showing the deciding moment. Fire an `ActivityToast`-style announcement of what happened (e.g., “Opponent cracked first!” / “You dodged in time!”). Update the running score. Auto-advance to the next round’s `Neutral` phase after the recap, or to `MatchResult` if a player has reached 3 round wins.

MatchResult phase: - Final score screen, winner declaration, rematch / share clip / return to Play tab options. This is also the trigger point for the post-match optional rewarded-ad offer (Section 11) and for saving match history/stats to Firestore.

8.5 Networking/signaling detail

- Use a per-match Realtime Database node (`/matches/{matchId}/events`) as an append-only event log: each client writes small event objects (`{type, timestamp, playerId}`) and both clients listen for new children. Avoid writing full state snapshots on every frame — event-sourcing this keeps payload tiny and avoids race conditions on shared mutable state.
- All authoritative timing decisions (cue-fire time, false-start eligibility, dodge-window evaluation) must be computed against **server timestamps** (`ServerValue.timestamp` or equivalent), never client `DateTime.now()`, to prevent a client with a fast/slow local clock from gaining an unfair advantage.
- Handle disconnects explicitly: use Realtime Database’s `onDisconnect()` hooks to mark a player as disconnected if their connection drops mid-match, triggering the reconnect-grace-period UX described in Blueprint Section 5.

8.6 Gesture engine integration

- `core/gesture_engine/` exposes a stream of blendshape values (`browInnerUp`, `jawOpen`, `smile-curvature`) sampled at a steady rate (target ~30fps to match the performance floor).
- The `duel` feature’s domain layer subscribes to this stream and applies the thresholding logic described above **on an isolate**, emitting clean, debounced semantic events (`fireDetected`, `dodgeDetected`, `crackDetected`) upward to the state machine — the state machine itself should never touch raw blendshape numbers, only these semantic events. This separation keeps the 300-line rule achievable and keeps game logic testable without a real camera (you can unit test the state machine by feeding it fake semantic events).

8.7 Tests required for this section specifically

- Unit tests for the round state machine covering: normal fire/dodge resolution, successful dodge, false start (both players), crack at every phase, simultaneous crack draw, round timeout draw, full best-of-5 match completion.
- A fake/mock signaling layer so these tests run with no real network or Firebase dependency.

9. Friends tab

- Friends list: online/offline indicator, avatar, display name, tap for actions (challenge to a private match, report, block, unfriend).
- Add friend: via shareable invite link (deep link opens app directly to a friend-request confirmation) or via the `PinCodeEntry` component (Blueprint Section 3) for manual code entry.
- Incoming friend requests: accept/decline list, badge count on the tab icon.
- Report/block: reporting requires a reason selection (short predefined list, e.g., “Inappropriate username,” “Harassment,” “Other” with optional text) submitted to a Firestore `reports` collection for later moderation review — do not build an automated moderation pipeline for v1, just reliable reporting capture. Blocking immediately prevents future matchmaking pairing and hides the blocked user from all lists, enforced both client-side (immediate UX) and server-side (Cloud Function or security rule check at matchmaking time, so a block can’t be bypassed by a modified client).

10. Profile tab

- Header: avatar, display name, level/tier bar.
- `StatTile` grid: win streak, total matches, friends count, win rate.
- Leaderboard entry point → `PodiumLeaderboard` component, global ranking by a defined score metric (document the metric explicitly, e.g., total round wins, in a code comment — don’t leave it implicit).
- Subscription section: current tier (Free / Face Off Plus), “Manage Subscription” (routes to platform subscription management or RevenueCat’s customer center if available), “Restore Purchases” button (required by App Store guidelines for any app with paid content).
- Cosmetics/inventory: owned skins/effects, equip/unequip.
- Settings: notification toggles, sign out, delete account flow (must actually delete the Firestore profile and Firebase Auth account, not just sign out — Apple requires real account deletion for App Store approval).
- FAQ & support: static content screen plus a support contact action (mailto or in-app form — pick one and implement it fully, don’t leave a dead-end button).

11. Monetization — RevenueCat integration

- Initialize RevenueCat SDK at app start (after auth, tied to the Firebase UID as the RevenueCat app user ID).
- **Cosmetic store (one-time purchases):** avatar skins, victory animations, themed match packs. Product catalog defined in RevenueCat dashboard, fetched via `Offerings`, rendered in a `monetization/presentation/store` screen accessible from Profile.
- **Face Off Plus (subscription):** HD/watermark-free clip export, unlimited private rooms, early access to new cosmetics. Paywall screen triggered from Profile’s subscription section and from any gated feature’s natural touchpoint (e.g., tapping “Export HD” when not subscribed shows the paywall, not a dead button).
- **RevenueCat Ads:** verify current integration approach against RevenueCat’s live documentation before implementing (their Ads product may have specific SDK requirements beyond the core purchases SDK — do not assume it’s bundled). Implement exactly two placements:
 1. Lobby banner tile on the Play tab (Blueprint Section 4).
 2. Optional rewarded ad on the match-result screen for bonus coins, opt-in only, never pre-selected or auto-played.
- No ad unit may appear during an active match, during onboarding, or during the auth flow.

12. Offline & connectivity handling

Implement exactly as specified in Blueprint Section 5: `connectivity_plus` stream → global provider → bottom sheet (slide-up, Lottie illustration, retry button, auto-dismiss on reconnect) → in-match reconnect grace period with `ActivityToast` messaging → graceful forfeit after timeout. Write this as its own testable unit (`core/connectivity/`) so the duel feature can consume a simple `isOnline` stream/provider rather than reimplementing detection logic.

13. Performance pass (do this after everything above works functionally)

- Profile the app on a real mid-range Android device, not just a simulator.
 - Confirm gesture pipeline sustains ~30fps without dropping UI frames — use Flutter DevTools' performance overlay, not guesswork.
 - Verify `RepaintBoundary` placement actually reduces rebuild scope (check via the widget rebuild highlighting in DevTools).
 - Implement the device-capability tiering described in Blueprint Section 6 if it isn't already in place, and confirm visual effects scale down correctly on a simulated/low-tier device profile.
-

Definition of done

Do not consider this build “finished” until: every section above compiles and runs, the Section 8 test suite passes in full, `dart format --line-length=120` produces no diff, no file exceeds 300 lines, no raw hex color exists outside `core/theme/`, and a full match can be played end-to-end on two physical devices with no crashes.